Lists

Many values under one name: make a list, read items by position, change, add, remove and search.

THE QUESTION

How do you keep many values, such as every score of a season, under one name?

THE INTUITION

A list is a row of numbered boxes under one name. You reach a box by its position, counted from 0. Unlike a string, a list can be changed: boxes can be refilled, added and removed.

KEY TAKEAWAY

Valid indexes run from 0 to `len(x) - 1`, or from −1 backwards. `append`, `sort` and `remove` change the list itself and give back `None`.

make: [90, 82] → read: x[1] → change: x[1] = 85 → add and remove → search: in, index → walk: while i < len(x)

1 Square brackets make a list

```
>>> scores = [90, 82, 83, 100, 102]
>>> scores
[90, 82, 83, 100, 102]
>>> empty = []
>>> print(empty, len(empty), list())
[] 0 []
>>> student = ["Alice", 80.5, "A", True]
>>> list("abc")
['a', 'b', 'c']
```

student: four types in one list

0	1	2	3
'Alice'	80.5	'A'	True
str	float	str	bool

len(student) is 4

list("abc"): an item per character

'a'	'b'	'c'

A **list** holds values in order, in square brackets with commas between them; each value is an **item**. One list can mix types. `[]` and `list()` are **empty** lists; `list("abc")` makes an item per character. `len(x)` counts the items, as for strings. Printing a list shows its brackets, and quotes around strings.

2 Positions count from 0, or from −1 at the back

index from the front	0	1	2	3	4	5
scores	90	82	83	100	102	?
from the back	-5	-4	-3	-2	-1	no item

```
scores[0] → 90                scores[-1] → 102
scores[3] → 100               scores[-5] → 90
len(scores) → 5               scores[len(scores) - 1] → 102
scores[5] → IndexError: list index out of range
```

An item's position is its **index**, in square brackets after the name. The first item is `scores[0]`, so the last is `scores[len(scores) - 1]`, or simply `scores[-1]`: negative indexes count from the end. Any index outside the boxes raises an **IndexError**. A run of items, such as `scores[1:3]`, is on the slicing sheet.

3 A list can change; a string cannot

`scores[1] = 85` refills box 1

90	~~82~~	83	100	102	→	90	85	83	100	102

the same list, one box changed

`word = "cat"`

'c'	'a'	't'

`word[0] → 'c'`
`word[0] = "b"` → error, below

`chars = list(word); chars[0] = "b"`

'b'	'a'	't'

```
TypeError: 'str' object does not support item assignment
```

`scores[1] = 85` refills box 1 of the same list: lists are **mutable** (changeable). On the original scores, a while loop doing `scores[i] = scores[i] + 4` for each `i` gives `[94, 86, 87, 104, 106]`. Strings are **immutable**: `word[0]` can be read, not assigned. Make a new string (`word = "b" + "at"`) or change a list of its characters. `b = scores` gives the same list a second name, not a copy: see the Python memory sheet.

4 `append`, `insert` and `extend` add items

`x = [1, 2, 3]`	1	2	3		
`x.append(4)`	1	2	3	4	added at the end
`x.insert(1, 9)`	1	9	2	3	4 — 9 goes in at index 1; 2, 3, 4 move right
`a.append([3, 4])`	1	2	[3, 4]		`[1, 2, [3, 4]]` len(a) is 3: a list inside a list
`b.extend([3, 4])`	1	2	3	4	`[1, 2, 3, 4]` len(b) is 4: each item added

`[1, 2] + [3]` → 1 2 3 `[0] * 3` → 0 0 0 each makes a new list

A **method** is a function called on a value with a dot. `x.append(v)` adds `v` at the end; `x.insert(i, v)` puts it at index `i`. Both change `x` and give back `None`: `print(x.append(5))` prints `None`. Appending a list adds it as one item; `extend` adds its items one by one. `+` joins lists and `*` repeats one.

5 `pop` and `remove` take items out

`x = [10, 20, ...]`	10	20	30	20	40
`last = x.pop()`	10	20	30	20	~~40~~ — gives back 40, last is 40
`first = x.pop(0)`	~~10~~	20	30	20	gives back 10, the rest move left
`x.remove(20)`	~~20~~	30	20		only the first 20 goes; the other stays

```
x.remove(99) → ValueError: list.remove(x): x not in list
[].pop()     → IndexError: pop from empty list
```

`x.pop()` removes the last item and gives it back, so you can keep it in a name; `x.pop(i)` does the same at index `i`. `x.remove(v)` searches by value and removes only the first match. A missing value raises a **ValueError**, so test with `in` first. An empty list has nothing to pop: IndexError.

6 `in`, `index` and `count` search a list

	0	1	2	3	4
rolls	3	6	2	6	5

index(6) finds the first 6, at index 1; count(6) is 2

```
6 in rolls      → True          rolls.index(6)  → 1
4 in rolls      → False         rolls.count(6)  → 2
4 not in rolls  → True          rolls.count(4)  → 0
rolls.index(4)  → ValueError: 4 is not in list
```

`v in x` is True when some item equals `v`; `v not in x` is the opposite. Both give a bool, ready for an `if`. `x.index(v)` gives the index of the first match, and a ValueError when there is none, so `if v in x:` should guard both `x.index(v)` and `x.remove(v)`. `x.count(v)` counts the matches.

Sort, total, find and tabulate

sort() versus sorted(), while loops over a list traced pass by pass, and lists of lists.

7 sort() versus sorted()

```
1 nums = [3, 1, 2]
2 new = sorted(nums)
3 print(nums, new)
```
prints `[3, 1, 2] [1, 2, 3]`

nums → 3 1 2 untouched
new → 1 2 3 a new list

```
1 nums = [3, 1, 2]
2 result = nums.sort()
3 print(nums, result)
```
prints `[1, 2, 3] None`

nums → 1 2 3 sorted in place
result → None sort() gives back None

```
1 nums = [3, 1, 2]
2 nums = nums.sort()
3 print(nums)
```
prints `None`

1 2 3 sorted, then lost: no name points here
nums → None nums now names None

`sorted(nums)` gives back a new sorted list and leaves `nums` alone. `nums.sort()` sorts `nums` itself and, like `append`, gives back `None`, so `nums = nums.sort()` loses the list. `reverse=True` puts the largest first: `[3, 2, 1]`. Strings sort capitals first: `['Fig', 'apple', 'pear']`.

8 Walk a list by index: totals and counts

scores = [90, 82, 83, 100, 102]

```
1 total = 0
2 above = 0
3 i = 0
4 while i < len(scores):
5     total = total + scores[i]
6     if scores[i] > 95:
7         above = above + 1
8     i = i + 1
9 print(total, above)
10 print(total / len(scores))
```

i	scores[i]	> 95	total	above
0	90	False	90	0
1	82	False	172	0
2	83	False	255	0
3	100	True	355	1
4	102	True	457	2
5	stop		457	2

at i = 5, i < len(scores) is False
prints `457 2` then `91.4`
blue: accumulator · teal: counter

Run `i` from 0 while `i < len(scores)`. `total` is an **accumulator**; `above` is a **counter** behind an `if`. `sum`, `max` and `min` give 457 102 82 at once. To build a list, `append` to `[]` in the loop: adding 4 to each score gives `[94, 86, 87, 104, 106]`. The for loops sheet walks a list with less code.

9 The highest value, and the first match

the highest score

```
1 best = scores[0]
2 i = 1
3 while i < len(scores):
4     if scores[i] > best:
5         best = scores[i]
6     i = i + 1
7 print(best)
```

i	scores[i]	> best	best after
1	82	False	90
2	83	False	90
3	100	True	100
4	102	True	102
5	stop		102

best starts at 90; prints 102

the first score above 95

```
1 i = 0
2 while i < len(scores) and scores[i] <= 95:
3     i = i + 1
4 print(i, i < len(scores))
```
prints `3 True` (no match: 5 False)

i	scores[i]	<= 95
0	90	True
1	82	True
2	83	True
3	100	False

Keep the best so far; replace it when an item beats it. Start at the first item: a guess of `-1` gives -1 for `[-5, -3, -8]`, a value not in the list. To find a match, move `i` on while the item does not match; `i < len(scores)` goes first, so `scores[i]` never runs past the end (the short-circuit sheet). The break sheet uses `break`.

10 A list of lists is a table

marks: a list of 3 rows

marks → 0 → 70 80 100
marks → 1 → 100 90 89
marks → 2 → 100 71 43

the same values as a table

row	0	1	2	column
0	70	80	100	
1	100	90	89	row 1
2	100	71	43	

the outer list holds an arrow per row
`marks[1]` → `[100, 90, 89]`
`len(marks)` → `3`
`marks[1][2]` → `89`
`len(marks[0])` → `3`
column 1, by a loop over the rows: `col.append(marks[r][1])` gives `[80, 90, 71]`

Each row is one student's marks in three tests. `marks[1]` is a whole row; `marks[1][2]` is item 2 of it. `len(marks)` counts rows, `len(marks[0])` columns. `append` adds a row; a column needs a loop.

11 Common mistakes: what you wrote, what Python says, the fix

you wrote	the last line Python prints; why, and the fix
`nums = nums.sort()` `print(nums[0])`	`TypeError: 'NoneType' object is not subscriptable` `sort()` gave back `None`. Write `nums.sort()` alone; then `nums[0]` is 1
`x.extend(5, 6, 7)`	`TypeError: list.extend() takes exactly one argument (3 given)` `extend` takes one list: `x.extend([5, 6, 7])`
`x = x + 10`	`TypeError: can only concatenate list (not "int") to list` `+` joins two lists: `x = x + [10]`, or `x.append(10)`
`while i <= len(x):` `  print(x[i])`	`prints 5 6 7, then IndexError: list index out of range` the last index is `len(x) - 1`: with `i < len(x)` it prints 5 6 7
`marks[1, 2]`	`TypeError: list indices must be integers or slices, not tuple` one pair of brackets per level: `marks[1][2]` gives 89

12 Try it (answers in green)

1. `len(["ab", [1, 2, 3], []])`?
3: a list inside is one item.

2. `t = [4, 7, 1]`, `t.insert(1, 9)`, `t.pop()`. Now `t`?
`[4, 9, 7]`: pop took the 1.

3. `[0] * 3 + [1]`?
`[0, 0, 0, 1]`

4. `v = [5, 3, 5, 1]`, `v.remove(5)`. `v`, `v.index(5)`?
`[3, 5, 1]` 1: only the first 5 went.

5. `g = [[1, 2], [3, 4], [5, 6]]`: `g[2][0]`, `g[-1][-1]`, `len(g)`?
5, 6, 3

6. Count the negatives in `[3, -1, 0, -7, 2]` with a while loop.
2: `if vals[i] < 0: neg = neg + 1`

index 0 to len - 1, or −1 back · lists change, strings do not · append, sort, remove give back None

Source: the lists notebook (its scores, marks and loops), nipunbatra.github.io/comp22; names are placeholders. Every output, trace value and error message comes from running the code with Python 3.11.11. Next: the slicing sheet.